

Exercise 10

Advanced Methods for Regression and Classification

Kyzer Gerez

07.01.2026

Load the data `Diabetes` from the package `ROCit`. Our goal is to find a classification model for `diabetes`, based on the variable `dtest` – see help file. Convert this information to a binary variable. To simplify the task, use as predictor variables: `chol`, `stab.glu`, `hdl`, `bmi`, `age`, and `gender`. Afterwards, delete observations with missings, using `na.omit()`.

```
library(ROCit); data(Diabetes)
library(mgcv)
library(caret)

# convert dtest to binary variable
Diabetes$dtest <- ifelse(Diabetes$dtest == "+", TRUE, FALSE)

# select only required predictors
vars <- c("chol", "stab.glu", "hdl", "bmi", "age", "gender", "dtest")
Diabetes <- Diabetes[vars]

# delete obs with NAs
Diabetes <- na.omit(Diabetes)
```

Select randomly a training set of about 3/4 of the observations from each class, build the classification model, predict the group membership for the (remaining) test data and, as the final result, show the resulting confusion table and compute the misclassification rate.

```
set.seed(12427543)

# create 3/4 train and test sets
samp_size <- floor(3/4*nrow(Diabetes))
train_indx <- sample(1:nrow(Diabetes), samp_size)
train <- Diabetes[train_indx,]
test <- Diabetes[-train_indx,]
```

- (1) Fit a logistic regression model, see Exercise 7, and report the results.

```
# train logistic regression model
mod.log.reg <- glm(dtest ~., train, family="binomial")

# predict on test
log.reg.pred <- predict(mod.log.reg, test)

# convert from linear to binary scale
log.reg.pred <- ifelse(log.reg.pred <= 0, FALSE, TRUE)

# build confusion matrix
```

```
log.reg.confusion <- confusionMatrix(data=as.factor(log.reg.pred),
                                     reference=as.factor(test$dtest))
log.reg.confusion$table
```

```
##           Reference
## Prediction FALSE TRUE
##      FALSE    78    5
##      TRUE     2    11
```

```
# compute misclassification rate
log.reg.misclass <- (log.reg.confusion$table[2,1] + # FP
                    log.reg.confusion$table[1,2]) / # FN
                    (nrow(test))
print(paste("The misclassification rate for logistic regression is",
            round(log.reg.misclass, 3)))
```

```
## [1] "The misclassification rate for logistic regression is 0.073"
```

- (2) Use GAMs, implemented in the function `gam()` of the package `library(mgcv)`, with the argument `family="binomial"`.

- (a) The smooth functions in GAMs can be defined for every variable by `s(variable)`, see also course notes. It might not make sense to use smooth functions for all variables, for sure not for factor variables. Now compute the GAM based on your chosen “formula”.

```
# train GAM
gam.object <- gam(dtest~s(chol)+s(stab.glu)+s(hdl)+s(bmi)+s(age)+gender,
                 data=train,
                 family="binomial")
```

- (b) Which variables are significant in the model? How complex are the smooth functions (in terms of the effective degrees of freedom)?

```
summary(gam.object)
```

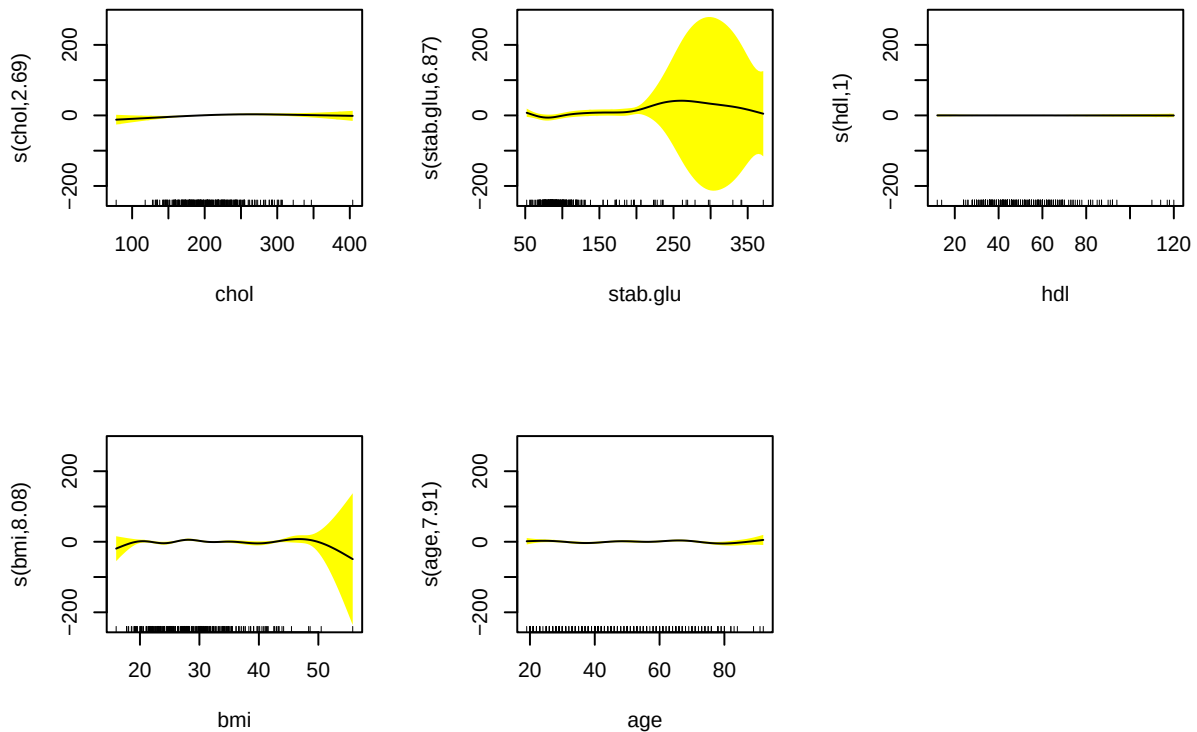
```
##
## Family: binomial
## Link function: logit
##
## Formula:
## dtest ~ s(chol) + s(stab.glu) + s(hdl) + s(bmi) + s(age) + gender
##
## Parametric coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept) -7.8781     4.3089  -1.828  0.0675 .
## gendermale   0.1008     1.1609   0.087  0.9308
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Approximate significance of smooth terms:
##              edf Ref.df Chi.sq p-value
## s(chol)       2.686  3.412  8.801  0.0450 *
## s(stab.glu)   6.868  7.081 18.444  0.0104 *
## s(hdl)        1.000  1.000  0.002  0.9622
## s(bmi)        8.079  8.510 13.110  0.1298
## s(age)        7.906  8.635 11.997  0.1839
## ---
```

```
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## R-sq.(adj) =  0.829   Deviance explained = 85.2%
## UBRE = -0.67783   Scale est. = 1           n = 287
```

The variables `chol` and `stab.glu` are significant. The smooth functions for the variables `stab.glu`, `bmi`, and `age` are rather complex, having about 7-8 effective degrees of freedom. In contrast, `hdl` only has 1 effective degree of freedom, so linear.

- (c) Plot the explanatory variables against their smoothed values as they are used in the model. You can simply use: `plot(gam.object, page=1, shade=TRUE, shade.col="yellow")` How can you interpret these plots?

```
plot(gam.object, page=1, shade=TRUE, shade.col="yellow")
```



The plots show the estimates for each smooth term. The yellow band indicates the 95% confidence interval for the smooth estimate. The ticks on the x-axis represent each datapoint for that predictor. Sparse regions generally show larger CIs since the extrapolation risk is greater. The y-axis shows the effect of the predictor on the response.

- (d) Report the confusion table and the misclassification error for the test set. How does this compare to (1)?

```
# predict on test
gam.pred <- predict(gam.object, test)

# convert from linear to binary scale
gam.pred <- ifelse(gam.pred <= 0, FALSE, TRUE)

# build confusion matrix
gam.confusion <- confusionMatrix(data=as.factor(gam.pred),
                                  reference=as.factor(test$dtest))

gam.confusion$table
```

```
##           Reference
## Prediction FALSE TRUE
##      FALSE      69      6
##      TRUE       11     10

# compute misclassification rate
gam.misclass <- (gam.confusion$table[2,1] + # FP
  gam.confusion$table[1,2]) / # FN
  (nrow(test))
print(paste("The misclassification rate for GAM is",
  round(gam.misclass, 3)))
```

```
## [1] "The misclassification rate for GAM is 0.177"
```

GAM performed significantly worse than logistic regression due to having a higher false positive rate.

- (e) The complexity of the smooth functions could be controlled by the parameter `k` within `s()`, which allows to set an upper bound for the effective degrees of freedom. If this makes sense, try it out and check if the results in (2)(b) and (d) change.

```
# train GAM
gam.object.mod <- gam(dtest~s(chol)+s(stab.glu,k=3)+s(hdl)+s(bmi)+s(age)+gender,
  data=train,
  family="binomial")

# show model
summary(gam.object.mod)
```

```
##
## Family: binomial
## Link function: logit
##
## Formula:
## dtest ~ s(chol) + s(stab.glu, k = 3) + s(hdl) + s(bmi) + s(age) +
##      gender
##
## Parametric coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept) -3.796138   0.621325  -6.110 9.98e-10 ***
## gendermale   0.001311   0.625050   0.002   0.998
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Approximate significance of smooth terms:
##              edf Ref.df Chi.sq p-value
## s(chol)       3.308  4.155 11.901  0.0191 *
## s(stab.glu)   2.000  2.000 40.448 <2e-16 ***
## s(hdl)        1.000  1.000  0.003  0.9560
## s(bmi)        1.000  1.000  0.128  0.7203
## s(age)        2.557  3.242  4.332  0.2599
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## R-sq.(adj) =  0.648   Deviance explained = 63.9%
## UBRE = -0.61703   Scale est. = 1           n = 287
```

```

# predict on test
gam.mod.pred <- predict(gam.object.mod, test)

# convert from linear to binary scale
gam.mod.pred <- ifelse(gam.mod.pred<=0, FALSE, TRUE)

# build confusion matrix
gam.mod.confusion <- confusionMatrix(data=as.factor(gam.mod.pred),
                                     reference=as.factor(test$dtest))
gam.mod.confusion$table

##           Reference
## Prediction FALSE TRUE
##      FALSE      76    5
##      TRUE       4   11

# compute misclassification rate
gam.mod.misclass <- (gam.mod.confusion$table[2,1] + # FP
                    gam.mod.confusion$table[1,2]) / # FN
                    (nrow(test))
print(paste("The misclassification rate for GAM with set EDoF is",
            round(gam.mod.misclass, 3)))

```

```
## [1] "The misclassification rate for GAM with set EDoF is 0.094"
```

I tested different values of `k` for the variables `chol` and `stab.glu` since they were significant in the original GAM. I found that setting `s(stab.glu,k=3)` yielded a much better model with a misclassification rate of 9.4%, though this is still higher than the logistic regression model. Worth noting is that the modified model now lists `stab.glu` as very significant.